Rust逆向之其他主题

介绍一下Rust中的生命周期，泛型和闭包。

1.生命周期

为了保证程序运行的安全，Rust编译器要求被引用变量的生命周期，长于引用该变量的变量的声明周期。在大多数时候，生命周期都是隐式，且可以按照下面的三条规则被推到出来。

> - 每一个引用参数都会拥有自己的生命周期参数
> - 当只存在一个输入生命周期参数时，这个生命周期会被赋予给所有输出生命周期参数
> - 当拥有多个输入生命周期参数，而其中一个是&self或&mut self时，self的生命周期会被赋予给所有的输出生命周期参数

但是，如果遇到下面的代码这样，引用的生命周期可能以不同的方式互相关联的时候，就需要手动标注生命周期。因为按照上面三条规则的第一个，s1和s2的生命周期分别是a和b，但是无法推出返回值的生命周期。

```
fn test_lifetime(s1: &str, s2: &str) -> &str {
    if s1.len() > s2.len() {
        s1
    } else {
        s2
    }
}
```

此时，编译这段代码会出现下面的错误。

```
error[E0106]: missing lifetime specifier
 --> src\main.rs:6:41
  |
6 | fn test_lifetime(s1: &str, s2: &str) -> &str {
  |                      ----      ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature
does not say whether it is borrowed from `s1` or `s2`
```

这个时候，要通过编译，就要像下面这样进行生命周期标记：

```rust
fn test_lifetime<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1.len() > s2.len() {
        s1
    } else {
        s2
    }
}
```

从汇编层面看，对于函数test_lifetime的调用，这里可直接用&str类型的变量作为参数没有区别：

```
.text:0000000140001150 ; void __fastcall demo::main()
.text:0000000140001150 demo__main      proc near
.text:0000000140001150
.text:0000000140001150                 sub     rsp, 48h
.text:0000000140001154                 lea     rax, aAaabbbbcalledO ; "aaabbbbcalled
`Option::unwrap()` on a `"...
.text:000000014000115B                 mov     [rsp+28h], rax
.text:0000000140001160                 mov     qword ptr [rsp+30h], 3
.text:0000000140001169                 lea     rax, aAaabbbbcalledO+3 ; "bbbbcalled
`Option::unwrap()` on a `Non"...
.text:0000000140001170                 mov     [rsp+38h], rax
.text:0000000140001175                 mov     qword ptr [rsp+40h], 4
.text:000000014000117E                 lea     rcx, aAaabbbbcalledO ; "aaabbbbcalled
`Option::unwrap()` on a `"...
.text:0000000140001185                 mov     edx, 3
.text:000000014000118A                 lea     r8, aAaabbbbcalledO+3 ; "bbbbcalled
`Option::unwrap()` on a `Non"...
.text:0000000140001191                 mov     r9d, 4
.text:0000000140001197                 call    demo__test_lifetime
.text:000000014000119C                 nop
.text:000000014000119D                 add     rsp, 48h
.text:00000001400011A1                 retn
.text:00000001400011A1 demo__main      endp
```

不仅如此，test_lifetime函数也一样看不出生命周期标注的痕迹。所以，生命周期标注并不会对被标注对象做任何的操作。生命周期的检查，就只是编译器层面的检查。

```
.text:00000001400011B0 ; ref$<str$> *__fastcall demo::test_lifetime(ref$<str$>
*result, ref$<str$>, ref$<str$>)
.text:00000001400011B0 demo__test_lifetime proc near
.text:00000001400011B0
.text:00000001400011B0                 sub     rsp, 78h
.text:00000001400011B4                 mov     [rsp+38h], r9
.text:00000001400011B9                 mov     [rsp+30h], r8
.text:00000001400011BE                 mov     [rsp+20h], rdx
.text:00000001400011C3                 mov     [rsp+40], rcx
.text:00000001400011C8                 mov     [rsp+58h], rcx
.text:00000001400011CD                 mov     [rsp+60h], rdx
.text:00000001400011D2                 mov     [rsp+68h], r8
.text:00000001400011D7                 mov     [rsp+70h], r9
.text:00000001400011DC                 call
_ZN4core3str21_$LT$impl$u20$str$GT$3len17hafc7720b9f18fb92E ;
core::str::_$LT$impl$u20$str$GT$::len::hafc7720b9f18fb92
.text:00000001400011E1                 mov     rcx, [rsp+30h]
.text:00000001400011E6                 mov     rdx, [rsp+38h]
.text:00000001400011EB                 mov     [rsp+40h], rax
.text:00000001400011F0                 call
_ZN4core3str21_$LT$impl$u20$str$GT$3len17hafc7720b9f18fb92E ;
core::str::_$LT$impl$u20$str$GT$::len::hafc7720b9f18fb92
.text:00000001400011F5                 mov     rcx, rax
.text:00000001400011F8                 mov     rax, [rsp+40h]
.text:00000001400011FD                 cmp     rax, rcx
.text:0000000140001200                 ja      short loc_140001218
.text:0000000140001202                 mov     rax, [rsp+38h]
.text:0000000140001207                 mov     rcx, [rsp+30h]
.text:000000014000120C                 mov     [rsp+48h], rcx
.text:0000000140001211                 mov     [rsp+50h], rax
.text:0000000140001216                 jmp     short loc_14000122C
.text:0000000140001218 ; ---------------------------------------------------------------
--------------
.text:0000000140001218
.text:0000000140001218 loc_140001218:                          ; CODE XREF:
demo__test_lifetime+50↑j
.text:0000000140001218                 mov     rax, [rsp+20h]
.text:000000014000121D                 mov     rcx, [rsp+28h]
.text:0000000140001222                 mov     [rsp+48h], rcx
.text:0000000140001227                 mov     [rsp+50h], rax
.text:000000014000122C
.text:000000014000122C loc_14000122C:                          ; CODE XREF:
demo__test_lifetime+66↑j
.text:000000014000122C                 mov     rax, [rsp+48h]
.text:0000000140001231                 mov     rdx, [rsp+50h]
.text:0000000140001236                 add     rsp, 78h
.text:000000014000123A                 retn
.text:000000014000123A demo__test_lifetime endp
```

生命周期标注也常见于结构体成员，但因为是编译器层面的检查，汇编层面看不出区别，这里就不放例子展示了。

2.泛型

Rust中的泛型，和C/C++中的泛型是一样的作用。下面是一个简单的例子：

```
fn test() {
    let x = get_value(5);
    let y = get_value('a');
    let z = get_value("Hello");
}

fn get_value<T>(value: T) -> T {
    value
}
```

test函数反汇编如下，这里可以看到，编译器会生成三个不同的get_value函数，然后分别传入参数调用这三个函数，将返回值赋值给变量。

```
.text:0000000140001140 demo__test      proc near
.text:0000000140001140
.text:0000000140001140                 sub     rsp, 38h
.text:0000000140001144                 mov     ecx, 5          ; int
.text:0000000140001149                 call    _ZN4demo9get_value17h29967adbac8f8da7E
; demo::get_value::h29967adbac8f8da7
.text:000000014000114E                 mov     [rsp+20h], eax  ; 为变量x赋值
.text:0000000140001152                 mov     ecx, 'a'
.text:0000000140001157                 call    _ZN4demo9get_value17h3d5ea2a71b7f29b9E
; demo::get_value::h3d5ea2a71b7f29b9
.text:000000014000115C                 mov     [rsp+24h], eax  ; 为变量y赋值
.text:0000000140001160                 lea     rcx, aHellocalledOpt ; "Hellocalled
`Option::unwrap()` on a `No"...
.text:0000000140001167                 mov     edx, 5
.text:000000014000116C                 call    _ZN4demo9get_value17h91425f494b07a875E
; demo::get_value::h91425f494b07a875
.text:0000000140001171                 mov     [rsp+28h], rax  ; 为变量z赋值
.text:0000000140001176                 mov     [rsp+30h], rdx
.text:000000014000117B                 add     rsp, 38h
.text:000000014000117F                 retn
.text:000000014000117F demo__test      endp
```

而这三个函数，会根据不同的参数来执行相应的代码，返回相应的数据。

```
.text:00000001400011C0 ; int __fastcall demo::get_value::h29967adbac8f8da7(int)
.text:00000001400011C0 _ZN4demo9get_value17h29967adbac8f8da7E proc near
.text:00000001400011C0
.text:00000001400011C0                 push    rax
.text:00000001400011C1                 mov     eax, ecx
.text:00000001400011C3                 mov     [rsp+4], eax
.text:00000001400011C7                 pop     rcx
.text:00000001400011C8                 retn
.text:00000001400011C8 _ZN4demo9get_value17h29967adbac8f8da7E endp
```

```
.text:00000001400011D0 ; demo::get_value::h3d5ea2a71b7f29b9
.text:00000001400011D0 _ZN4demo9get_value17h3d5ea2a71b7f29b9E proc near
.text:00000001400011D0
.text:00000001400011D0                 push    rax
.text:00000001400011D1                 mov     eax, ecx
.text:00000001400011D3                 mov     [rsp+4], eax
.text:00000001400011D7                 pop     rcx
.text:00000001400011D8                 retn
.text:00000001400011D8 _ZN4demo9get_value17h3d5ea2a71b7f29b9E endp
```

```
.text:00000001400011E0 ; ref$<str$> *__fastcall
demo::get_value::h91425f494b07a875(ref$<str$> *result, ref$<str$>)
.text:00000001400011E0 _ZN4demo9get_value17h91425f494b07a875E proc near
.text:00000001400011E0
.text:00000001400011E0                 sub     rsp, 10h
.text:00000001400011E4                 mov     rax, rcx
.text:00000001400011E7                 mov     [rsp], rax
.text:00000001400011EB                 mov     [rsp+8], rdx
.text:00000001400011F0                 add     rsp, 10h
.text:00000001400011F4                 retn
.text:00000001400011F4 _ZN4demo9get_value17h91425f494b07a875E endp
```

所以，和C/C++一样。调用泛型函数，编译器会根据参数类型来生成相应的函数。下面是一个，结构体和方法中使用泛型的例子：

```rust
struct Test<T> {
    x: T,
    y: T,
}

impl <T> Test<T> {
    fn new(x: T, y: T) -> Self {
        Test {
            x,
            y,
        }
    }
}

fn test() {
    let t = Test::new(1, 2);
    let t1 = Test::new('a', 'b');
}
```

和上面类似，这里也是会根据参数，调用不同的new函数。获取返回值以后，会赋值到相应的变量中，代表了不同类型的结构体变量。

```
.text:0000000140001140 demo__test      proc near
.text:0000000140001140
.text:0000000140001140                 sub     rsp, 38h
.text:0000000140001144                 mov     ecx, 1          ; int
.text:0000000140001149                 mov     edx, 2          ; int
.text:000000014000114E                 call
_ZN4demo13Test$LT$T$GT$3new17h7ae79c26caf60359E ;
demo::Test$LT$T$GT$::new::h7ae79c26caf60359
.text:0000000140001153                 mov     [rsp+28h], eax  ; 为变量t赋值
.text:0000000140001157                 mov     [rsp+2Ch], edx
.text:000000014000115B                 mov     ecx, 'a'
.text:0000000140001160                 mov     edx, 'b'
.text:0000000140001165                 call
_ZN4demo13Test$LT$T$GT$3new17h28465a0ee2caa7ebE ;
demo::Test$LT$T$GT$::new::h28465a0ee2caa7eb
.text:000000014000116A                 mov     [rsp+30h], eax  ; 为变量t1赋值
.text:000000014000116E                 mov     [rsp+34h], edx
.text:0000000140001172                 add     rsp, 38h
.text:0000000140001176                 retn
.text:0000000140001176 demo__test      endp
```

所以在泛型这一块，Rust和C/C++差不多。都是会根据使用的数据类型，去生成相应的函数或者结构体。

3.闭包

下面是一个闭包的例子：

```
fn test() {
    let x = 5;

    let add_one = |value: i32| -> i32 { value + 1 };

    let y = add_one(x);
}
```

对应的反汇编如下，这里可以看到，在调用闭包的时候。除了会传入参数x，还会将栈中的一块地址作为第一个参数传递进去。

```
.text:0000000140001140 demo__test      proc near
.text:0000000140001140
.text:0000000140001140                 sub     rsp, 38h
.text:0000000140001144                 mov     dword ptr [rsp+28h], 5
.text:000000014000114C                 mov     dword ptr [rsp+30h], 5
.text:0000000140001154                 mov     edx, [rsp+30h]  ; int
.text:0000000140001158                 lea     rcx, [rsp+2Fh]  ;
demo::test::closure_env$0 *
.text:000000014000115D                 call
_ZN4demo4test28_$u7b$$u7b$closure$u7d$$u7d$17hd8a5af8e62bf6827E ;
demo::test::_$u7b$$u7b$closure$u7d$$u7d$::hd8a5af8e62bf6827
.text:0000000140001162                 mov     [rsp+34h], eax  ; 将返回值赋给y
.text:0000000140001166                 add     rsp, 38h
.text:000000014000116A                 retn
.text:000000014000116A demo__test      endp
```

而闭包函数内部，则看不出有什么区别。

```
.text:00000001400011B0 ; int __fastcall
demo::test::_$u7b$$u7b$closure$u7d$$u7d$::hd8a5af8e62bf6827(demo::test::closure_env$0
*, int)
.text:00000001400011B0 _ZN4demo4test28_$u7b$$u7b$closure$u7d$$u7d$17hd8a5af8e62bf6827E
proc near
.text:00000001400011B0
.text:00000001400011B0
.text:00000001400011B0                 sub     rsp, 38h
.text:00000001400011B4                 mov     [rsp+28h], rcx
.text:00000001400011B9                 mov     [rsp+34h], edx
.text:00000001400011BD                 inc     edx             ; edx=x+1
.text:00000001400011BF                 mov     [rsp+24h], edx  ; 保存x+1
.text:00000001400011C3                 seto    al
.text:00000001400011C6                 test    al, 1
.text:00000001400011C8                 jnz     short loc_1400011D3 ; 整型溢出则跳转
.text:00000001400011CA                 mov     eax, [rsp+24h]  ; eax=x+1
.text:00000001400011CE                 add     rsp, 38h
.text:00000001400011D2                 retn
.text:00000001400011D3 ; ---------------------------------------------------------------
--------------
.text:00000001400011D3
.text:00000001400011D3 loc_1400011D3:                          ; CODE XREF:
demo::test::_$u7b$$u7b$closure$u7d$$u7d$::hd8a5af8e62bf6827+18↑j
.text:00000001400011D3                 lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:00000001400011DA                 lea     r8, off_140019360 ; "src\\main.rs"
.text:00000001400011E1                 mov     edx, 1Ch
.text:00000001400011E6                 call
_ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:00000001400011E6 ; ---------------------------------------------------------------
--------------
.text:00000001400011EB                 db 0CCh
.text:00000001400011EB _ZN4demo4test28_$u7b$$u7b$closure$u7d$$u7d$17hd8a5af8e62bf6827E
endp
```

下面的例子，是一个用到环境变量的闭包。

```rust
fn test() {
    let x = 5;
    let y = 1;

    let add_one = |value: i32| -> i32 { value + x };

    let z = add_one(y);
}
```

test函数反汇编代码如下，此时调用闭包函数的时候。依然会将一个栈中地址和变量y作为参数传入，不过此时第一个参数保存的地址中，保存了环境变量x的地址。所以，此时第一个参数就是变量x的指针的指针。

```
.text:0000000140001140 demo__test      proc near
.text:0000000140001140
.text:0000000140001140                 sub     rsp, 48h
.text:0000000140001144                 mov     dword ptr [rsp+2Ch], 5 ; x=5
.text:000000014000114C                 mov     dword ptr [rsp+30h], 1 ; y=1
.text:0000000140001154                 mov     dword ptr [rsp+34h], 5
.text:000000014000115C                 lea     rax, [rsp+34h]  ; rax=保存数字5的地址
.text:0000000140001161                 mov     [rsp+38h], rax  ; 保存rax
.text:0000000140001166                 mov     dword ptr [rsp+40h], 1
.text:000000014000116E                 mov     edx, [rsp+40h]  ; edx=1
.text:0000000140001172                 lea     rcx, [rsp+38h]  ; ecx=rax
.text:0000000140001177                 call
_ZN4demo4test28_$u7b$$u7b$closure$u7d$$u7d$17h5840445f6ffdbcb7E ;
demo::test::_$u7b$$u7b$closure$u7d$$u7d$::h5840445f6ffdbcb7
.text:000000014000117C                 mov     [rsp+44h], eax  ; 将返回值赋值给z
.text:0000000140001180                 add     rsp, 48h
.text:0000000140001184                 retn
.text:0000000140001184 demo__test      endp
```

在闭包函数的反汇编中，就会根据传入的参数获取x的值，并将其与参数value相加，然后将相加的值作为返回值。

```
.text:00000001400011D0 ; int __fastcall
demo::test::_$u7b$$u7b$closure$u7d$$u7d$::h5840445f6ffdbcb7(demo::test::closure_env$0
*, int)
.text:00000001400011D0 _ZN4demo4test28_$u7b$$u7b$closure$u7d$$u7d$17h5840445f6ffdbcb7E
proc near
.text:00000001400011D0
.text:00000001400011D0
.text:00000001400011D0                 sub     rsp, 48h
.text:00000001400011D4                 mov     [rsp+30h], rcx
.text:00000001400011D9                 mov     rax, [rsp+30h]
.text:00000001400011DE                 mov     rax, [rax]
.text:00000001400011E1                 mov     [rsp+38h], rax
.text:00000001400011E6                 mov     [rsp+44h], edx
.text:00000001400011EA                 mov     rax, [rcx]      ; rax=变量x的地址
.text:00000001400011ED                 add     edx, [rax]      ; edx = value + x
.text:00000001400011EF                 mov     [rsp+2Ch], edx  ; 保存value+x
.text:00000001400011F3                 seto    al
.text:00000001400011F6                 test    al, 1
.text:00000001400011F8                 jnz     short loc_140001203 ; 整型溢出则跳转
.text:00000001400011FA                 mov     eax, [rsp+2Ch]  ; 将value+x作为返回值
.text:00000001400011FE                 add     rsp, 48h
.text:0000000140001202                 retn
.text:0000000140001203 ; ---------------------------------------------------------------
--------------
.text:0000000140001203
.text:0000000140001203 loc_140001203:                          ; CODE XREF:
demo::test::_$u7b$$u7b$closure$u7d$$u7d$::h5840445f6ffdbcb7+28↑j
.text:0000000140001203                 lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:000000014000120A                 lea     r8, off_140019360 ; "src\\main.rs"
.text:0000000140001211                 mov     edx, 1Ch
.text:0000000140001216                 call
_ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:0000000140001216 ; ---------------------------------------------------------------
--------------
.text:000000014000121B                 db 0CCh
.text:000000014000121B _ZN4demo4test28_$u7b$$u7b$closure$u7d$$u7d$17h5840445f6ffdbcb7E
endp
```

因此可以看出，闭包函数相比于普通函数，会多传入一个地址。通过这个地址，闭包函数可以找到，在函数中用到的环境变量。